Nome e Cognome: _____ **Matricola:** _____

Istruzioni: Il tempo a disposizione per completare la prova è di **45 minuti**. Durante la prova, non è consentito consultare materiale didattico, né interagire con dispositivi elettronici o colleghi. Nel caso di domande a risposta multipla con più risposte corrette, ciascuna risposta corretta indicata vale una corrispondente frazione del punteggio complessivo, mentre le risposte errate comportano una pari penalizzazione. In nessun caso, il punteggio complessivo relativo a una domanda sarà negativo.

Es. **1.** **(2 punti)** Quali dei seguenti sono modi validi per applicare regole CSS a una pagina HTML?

- [ ] A `<css href="stile.css">` nell'head
- [x] B Un elemento `<style>` con le regole al suo interno
- [ ] C Un commento HTML che contiene le regole
- [x] D `<link rel="stylesheet" href="stile.css">` nell'head
- [ ] E `<script src="stile.css">` nell'head
- [ ] F `<style src="stile.css">` senza contenuto

Risposte: B, D. Foglio esterno con `<link rel="stylesheet">`, elemento `<style>` nel documento e attributo `style` (stile inline). Il foglio esterno è da preferire: separa contenuto e presentazione e si condivide fra più pagine (e il browser lo tiene in cache). *(Dispensa: cap. 3, «Come includere il CSS in una pagina»)*

Es. **2.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
1  function contatore() {
2    let n = 0;
3    return () => ++n;
4  }
5  const a = contatore();
6  const b = contatore();
7  a(); a();
8  console.log(a(), b());
```

- [x] B `3 1`.
- [ ] A `3 4`.
- [ ] C Viene sollevato un `ReferenceError`.
- [ ] D `3 3`.
- [ ] E `1 1`.
- [ ] F `2 1`.

Risposta: B. Ogni chiamata di `contatore` crea un nuovo ambiente con la propria variabile `n`, e la funzione restituita lo ricorda (*closure*). `a` e `b` hanno contatori indipendenti: `a` arriva a 3, `b` a 1. *(Dispensa: cap. 5, «Funzioni annidate restituite»)*

Es. **3.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
1  function f() {
2    try {
3      console.log("A");
4      null.x;
5      console.log("B");
6    } catch (e) {
7      console.log(e instanceof TypeError, e instanceof Error);
8      return "C";
9    } finally {
10     console.log("D");
11   }
12 }
13 console.log(f());
```

- [ ] A `A, true true, C`.
- [ ] B `A, true true, C, D`.
- [ ] C `A, false true, D, C`.
- [ ] D `A, B, D, C`.
- [x] E `A, true true, D, C`.
- [ ] F `A, true false, D, C`.

Risposta: E. Leggere una proprietà di `null` solleva un `TypeError`, quindi `B` non viene stampata. Ogni errore predefinito eredita da `Error`. Il `return` nel `catch` viene eseguito, ma prima che la funzione restituisca davvero il valore viene eseguito il blocco `finally`. *(Dispensa: cap. 6, «try / catch / finally»)*

SOLUZIONI

Es. **4.** **(2 punti)** Qual è l'output in console dopo il caricamento completo del documento HTML riportato di seguito?

```
<script>
  var corso = "TW";
  let anno = 2026;
  function saluta() { return "ciao"; }
  console.log(window.corso, window.anno, typeof window.saluta);
</script>
```

- A `undefined undefined undefined`.
- **B** `TW undefined function`.
- C `undefined undefined function`.
- D Viene sollevato un `ReferenceError`.
- E `TW undefined undefined`.
- F `TW 2026 function`.

Risposta: B. `window` è l'oggetto globale: le funzioni dichiarate a livello globale e le variabili `var` ne diventano proprietà. Le variabili `let` e `const` globali vivono invece in uno scope separato e **non** diventano proprietà di `window`. *(Dispensa: cap. 7, «L'oggetto window»)*

Es. **5.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
async function leggi(ok) {
  if (!ok) throw new Error("rifiutata");
  return "dati";
}
async function main() {
  try {
    console.log(await leggi(true));
    console.log(await leggi(false));
    console.log("dopo");
  } catch (e) {
    console.log("errore:", e.message);
  }
}
main();
```

- A `dati`, `errore: rifiutata`, `dopo`.
- B `dati`, `rifiutata`, `dopo`.
- **C** `dati`, `errore: rifiutata`.
- D `errore: rifiutata`.
- E `dati`, `undefined`, `dopo`.
- F `dati`, poi viene sollevato un `Error`.

Risposta: C. Un errore sollevato in una funzione `async` diventa una Promise rifiutata; `await` su una Promise rifiutata **solleva** l'errore, che si cattura con un normale `try/catch`. L'istruzione dopo l'`await` fallito non viene eseguita. *(Dispensa: cap. 8, «async e await»)*

Es. **6.** **(2 punti)** Che cos'è un *socket*?

- **A** L'interfaccia software fra il livello applicativo e il livello di trasporto, attraverso cui un processo invia e riceve messaggi sulla rete.
- B Un messaggio di controllo scambiato durante l'*handshake* di TCP.
- C Il cavo che collega un host al primo router della rete.
- D Il processo del server web che resta in ascolto sulla porta 80.
- E Il protocollo di trasporto usato da HTTP al posto di TCP.
- F Un file di configurazione che associa i nomi degli host agli indirizzi IP.

Risposta: A. Il socket è la «cassetta postale» del processo: l'applicazione vi imbuca i messaggi, e l'infrastruttura di trasporto, controllata dal sistema operativo, li consegna al socket del destinatario. Dal lato del trasporto lo sviluppatore sceglie solo il protocollo (TCP o UDP) e pochi parametri. *(Dispensa: cap. 9, «Il socket»)*

SOLUZIONI

Es. **7.** **(2 punti)** Quali delle seguenti affermazioni su Node.js sono **vere**?

- [A] È un framework per scrivere applicazioni web, come Express.
- [**B**] È un ambiente di esecuzione JavaScript basato sul motore V8 di Google.
- [C] È un linguaggio di programmazione derivato da Java.
- [D] Mette a disposizione gli oggetti `window` e `document`.
- [E] Crea un nuovo thread per ogni richiesta HTTP che riceve.
- [F] Rende parallelo il codice JavaScript sui core della CPU.

Risposta: B. Node.js è un *runtime* JavaScript (motore V8) con I/O asincrono e non bloccante. Il nostro codice gira in un unico thread, l'event loop; l'I/O è delegato ai worker del runtime, e la sua callback torna all'event loop. Express è un framework costruito su Node.js; DOM e BOM non esistono fuori dal browser. *(Dispensa: cap. 11, «Che cos'è Node.js»)*

Es. **8.** **(2 punti)** Un'applicazione scritta per Express 4 dichiara la rotta `app.get("/web(-)?tech*", ...)`. Che cosa succede eseguendola con Express 5?

- [**A**] Il programma si ferma già all'avvio, quando la rotta viene dichiarata, con un errore sulla sintassi del percorso.
- [B] Funziona come in Express 4: corrisponde a `/webtech`, `/web-tech`, `/webtechs`…
- [C] La rotta viene ignorata in silenzio e tutte le richieste ricevono 404.
- [D] L'errore compare solo alla prima richiesta che corrisponde.
- [E] Express 5 la converte automaticamente in un'espressione regolare.
- [F] La rotta corrisponde solo al percorso letterale `/web(-)?tech*`.

Risposta: A. Express 5 non ammette più nei percorsi i caratteri speciali `?`, `+`, `*` e le parentesi: la dichiarazione solleva un `PathError` all'avvio. Le parti facoltative si scrivono con le graffe (`/web{-}tech`), una sequenza qualsiasi con un jolly con nome (`{*resto}`), oppure si usa un'espressione regolare. *(Dispensa: cap. 15, «Percorsi delle rotte»)*

Es. **9.** **(5 punti)** Si descrivano le parti di una URL, usando come esempio `https://www.example.org:8443/corsi/web?anno=2025#esami`.

Traccia della risposta.

- Schema `https`: il protocollo (HTTP dentro TLS).
- Host `www.example.org`: il nome di dominio, tradotto dal DNS in un indirizzo IP.
- Porta `8443`: il processo server sulla macchina; se omessa vale 443 per https e 80 per http.
- Percorso `/corsi/web`: la risorsa sul server.
- Query `anno=2025`: parametri `chiave=valore` separati da `&`; ancora `esami`: usata solo dal browser, non viene inviata al server.

(Dispensa: cap. 1, «URL: individuare una risorsa»)

Es. **10.** **(5 punti)** Quali limiti ha un sito composto da sole pagine statiche? Si spieghi, anche con un esempio, che cosa cambia quando le pagine vengono generate lato server.

Traccia della risposta.

- Una pagina statica è un file della *document root*: a ogni richiesta il server invia sempre lo stesso documento (JavaScript aggiunge dinamismo solo nel browser).
- Non si può personalizzare la pagina per l'utente, i contenuti che cambiano spesso vanno riscritti a mano e non si può *agire* in seguito a una richiesta (per esempio registrare un ordine).
- Con milioni di pagine la struttura comune si ripete in ogni file: per Wikipedia (circa 60 milioni di pagine) sono circa 9 TB di dati duplicati, e cambiare la barra di navigazione vorrebbe dire rigenerare tutti i file.
- Con la programmazione lato server il server genera la pagina al volo, in base ai parametri della richiesta e ai dati: la stessa URL può produrre risposte diverse (CGI, PHP, Node.js).

(Dispensa: cap. 10, «I limiti delle web app statiche» e «La programmazione lato server»)

SOLUZIONI

Es. **11.** **(5 punti)** Si descrivano le pseudo-classi funzionali `:is()`, `:where()`, `:not()` e `:has()`, con un esempio d'uso ciascuna, e le regole con cui contribuiscono alla specificità.

Traccia della risposta.

- `:is()` raggruppa selettori alternativi (es. `:is(header, main, footer) a em`); `:where()` fa lo stesso ma con specificità zero, utile per stili predefiniti facili da sovrascrivere.
- `:not()` esclude gli elementi che corrispondono ad almeno un argomento (es. `li:not(.selected, .active)`).
- `:has()` seleziona in base a discendenti o elementi collegati, con selettori relativi (es. `ol:has(.active)`, `li:has(+ .selected)`).
- Specificità: non contano come pseudo-classi; `:where()` vale zero; le altre valgono quanto il loro argomento più specifico, qualunque sia quello che seleziona l'elemento.

(Dispensa: cap. 14, «Selettori moderni»)

Griglia delle risposte

Es.	Risposta	Capitolo	Argomento
1	B, D	3	includere il CSS
2	B	5	closure
3	E	6	gestione degli errori
4	B	7	DOM e BOM
5	C	8	async e await
6	A	9	processi, socket e porte
7	B	11	che cos'è Node.js
8	A	15	routing
9	aperta	1	anatomia di una URL
10	aperta	10	pagine statiche e dinamiche
11	aperta	14	pseudo-classi funzionali

Punteggio massimo: 16 (chiuse) + 15 (aperte) = **31 punti**. Nelle domande con più risposte corrette ogni risposta giusta indicata vale una frazione dei punti e ogni risposta sbagliata indicata ne toglie altrettanti; il punteggio di una domanda non scende mai sotto zero.

SOLUZIONI